

ASSESSMENT TOOL - 01

FLOWGORITHM FLOWCHART & JAVA CODE ASSIGNMENTS

Question 1 — BMI Calculator and Classification

Question 2 — Checking if an Array is Sorted in Ascending Order

Question 3 — Food-Ordering System with Regular / Premium
Delivery Charges (including class inheritance relationship)

Flowchart Reference + Java Implementation for Assignment Submission

Course Code : CSA0924

Course Name : Java Programming

Name : M. Yogeshwaran

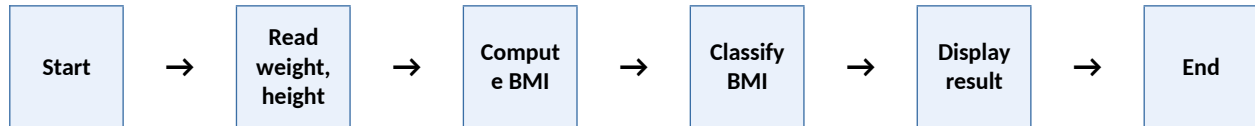
Register No. : 192524363

Question 1: BMI Calculator and Classification

Aim

To read a person's weight and height, compute the Body Mass Index (BMI), and classify the result into Underweight, Normal, Overweight or Obese categories using the standard WHO cut-offs.

Flow of Logic



Algorithm / Pseudocode

1. Start
2. Read weight (kg) and height (m).
3. Compute $BMI = \text{weight} / (\text{height} \times \text{height})$.
4. If $BMI < 18.5$, category = "Underweight".
5. Else if $BMI \leq 24.9$, category = "Normal".
6. Else if $BMI \leq 29.9$, category = "Overweight".
7. Else, category = "Obese".
8. Display BMI value and category.
9. Stop

Java Program

```
import java.util.Scanner;

public class BMICalculator {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter weight in kilograms: ");
        double weight = sc.nextDouble();
        System.out.print("Enter height in metres: ");
        double height = sc.nextDouble();

        double bmi = weight / (height * height);
        String category;

        if (bmi < 18.5) {
            category = "Underweight";
        } else if (bmi <= 24.9) {
            category = "Normal";
        } else if (bmi <= 29.9) {
            category = "Overweight";
        } else {
```

```
        category = "Obese";
    }

    System.out.println("Your BMI is: " + bmi);
    System.out.println("Category: " + category);
    sc.close();
}
}
```

Output

```
Enter weight in kilograms: 70
Enter height in metres: 1.75
Your BMI is: 22.857142857142858
Category: Normal
```

Result

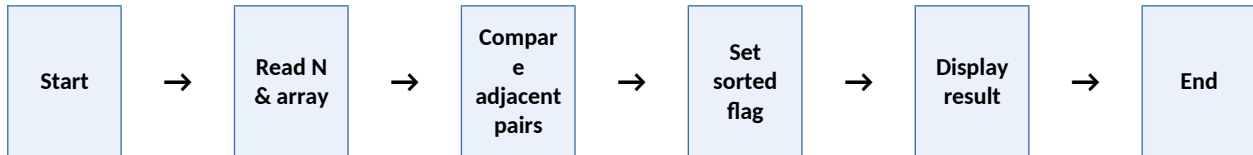
The program correctly computes $BMI = \text{weight} / \text{height}^2$ and classifies it using the WHO cut-offs (<18.5 Underweight, 18.5–24.9 Normal, 25–29.9 Overweight, ≥ 30 Obese). Using else if ensures only the first matching range is applied, giving an unambiguous single classification for any input.

Question 2: Checking if an Array is Sorted in Ascending Order

Aim

To read N numeric values into an array and determine whether the array is arranged in ascending (non-decreasing) order.

Flow of Logic



Algorithm / Pseudocode

1. Start
2. Read N, the number of elements.
3. Read array A[0..N-1].
4. Set sorted = true, i = 1.
5. While i < N: if A[i] < A[i-1], set sorted = false.
6. Increment i and repeat step 5 until i = N.
7. If sorted is true, display "Array is sorted"; else display "Array is NOT sorted".
8. Stop

Java Program

```
import java.util.Scanner;

public class CheckSortedArray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of elements N: ");
        int n = sc.nextInt();
        double[] a = new double[n];
        for (int i = 0; i < n; i++) {
            System.out.print("Enter A[" + i + "]: ");
            a[i] = sc.nextDouble();
        }

        boolean sorted = true;
        for (int i = 1; i < n; i++) {
            if (a[i] < a[i - 1]) {
                sorted = false;
            }
        }
    }
}
```

```
        if (sorted) {
            System.out.println("Array is sorted in ascending order");
        } else {
            System.out.println("Array is NOT sorted in ascending order");
        }
        sc.close();
    }
}
```

Output

```
Enter number of elements N: 5
Enter A[0]: 5
Enter A[1]: 10
Enter A[2]: 15
Enter A[3]: 20
Enter A[4]: 25
Array is sorted in ascending order
```

Result

The program scans every adjacent pair exactly once ($O(N)$ time, $O(1)$ extra space besides the array itself) and flips a boolean flag the moment an out-of-order pair is found. Because the loop does not stop early, all elements are still read, but the sorted flag correctly reflects the final ascending/not-ascending verdict for any input, including edge cases like $N = 0$ or $N = 1$ (trivially sorted).

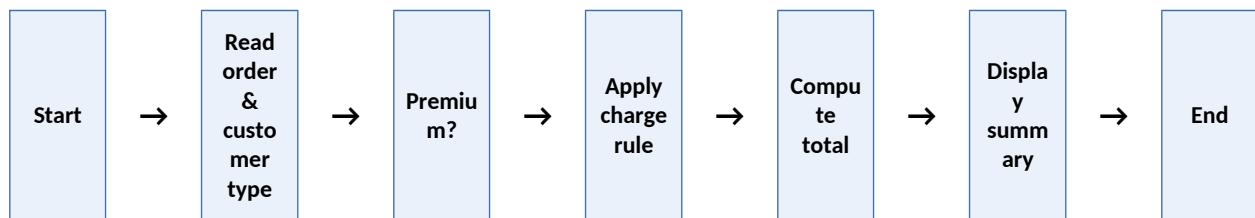
Question 3: Food-Ordering System with Regular / Premium Delivery Charges

Aim

To build an order-billing program that computes the total payable amount for a food order, applying a different delivery charge depending on whether the customer is Regular or Premium — first with a simple procedural version, then re-modelled using class inheritance.

Part A — Procedural Version

Flow of Logic



Algorithm / Pseudocode

1. Start
2. Read customer name, order amount, and customer type (Regular / Premium).
3. If customer type = "Premium", set deliveryCharge = 20 (discounted / free above minimum order).
4. Else, set deliveryCharge = 50 (standard flat rate).
5. Compute total = orderAmount + deliveryCharge.
6. Display customer name, order amount, delivery charge and total payable.
7. Stop

Java Program

```
import java.util.Scanner;

public class FoodOrder {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter customer name: ");
        String customerName = sc.nextLine();
        System.out.print("Enter order amount: ");
        double orderAmount = sc.nextDouble();
        sc.nextLine();
        System.out.print("Enter customer type (Regular / Premium): ");
        String customerType = sc.nextLine();

        double deliveryCharge;
        if (customerType.equalsIgnoreCase("Premium")) {
```

```

        deliveryCharge = 20;
    } else {
        deliveryCharge = 50;
    }

    double total = orderAmount + deliveryCharge;

    System.out.println("Customer: " + customerName);
    System.out.println("Order Amount: " + orderAmount);
    System.out.println("Delivery Charge: " + deliveryCharge);
    System.out.println("Total Payable: " + total);
    sc.close();
}
}

```

Output

```

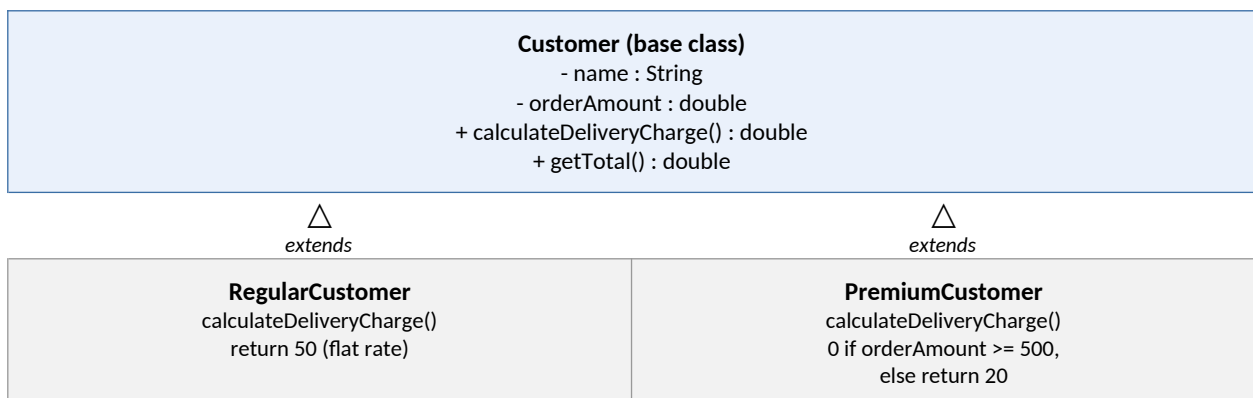
Enter customer name: Ramesh
Enter order amount: 300
Enter customer type (Regular / Premium): Regular
Customer: Ramesh
Order Amount: 300.0
Delivery Charge: 50.0
Total Payable: 350.0

```

Part B — Inheritance-Based Version

Regular and Premium customers share the same core order details (name, order amount) and differ only in how the delivery charge is calculated. This is a natural fit for inheritance: shared data and behaviour live in a base class, while each customer type overrides just the delivery-charge rule.

Class Diagram (Inheritance Relationship)



The hollow-triangle arrows point from each subclass up to the base class it inherits from (UML "is-a" relationship).

Algorithm / Pseudocode

1. Start

2. Define base class Customer with fields name, orderAmount and method calculateDeliveryCharge() returning 0 (to be overridden), plus getTotal() = orderAmount + calculateDeliveryCharge().
3. Define RegularCustomer extends Customer, overriding calculateDeliveryCharge() to return 50.
4. Define PremiumCustomer extends Customer, overriding calculateDeliveryCharge() to return 0 if orderAmount >= 500, else 20.
5. Read the customer type and instantiate the matching subclass (RegularCustomer or PremiumCustomer).
6. Read name and orderAmount into the created object.
7. Call getTotal() — this automatically invokes the correct overridden method via runtime polymorphism.
8. Display the total payable.
9. Stop

Java Program

```
class Customer {
    String name;
    double orderAmount;

    double calculateDeliveryCharge() {
        return 0;                // overridden by each subclass
    }
    double getTotal() {
        return orderAmount + calculateDeliveryCharge();
    }
}

class RegularCustomer extends Customer {
    @Override
    double calculateDeliveryCharge() {
        return 50;                // standard flat rate
    }
}

class PremiumCustomer extends Customer {
    @Override
    double calculateDeliveryCharge() {
        if (orderAmount >= 500) {
            return 0;                // free delivery above minimum
        } else {
            return 20;                // discounted rate
        }
    }
}

import java.util.Scanner;
```



```

public class FoodOrderMain {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Customer type (Regular/Premium): ");
        String custType = sc.nextLine();

        Customer c;
        if (custType.equalsIgnoreCase("Premium")) {
            c = new PremiumCustomer();
        } else {
            c = new RegularCustomer();
        }

        System.out.print("Enter name: ");
        c.name = sc.nextLine();
        System.out.print("Enter order amount: ");
        c.orderAmount = sc.nextDouble();

        System.out.println("Total Payable: " + c.getTotal());
        sc.close();
    }
}

```

Output

```

Customer type (Regular/Premium): Premium
Enter name: Anitha
Enter order amount: 600
Total Payable: 600.0

```

Result

Both versions produce the same billing logic, but the inheritance-based version demonstrates the object-oriented principles of inheritance (RegularCustomer and PremiumCustomer extend Customer), method overriding (each subclass supplies its own calculateDeliveryCharge()), and runtime polymorphism (getTotal() calls the correct overridden method automatically based on the actual object type, without any if-else on customer type inside getTotal() itself). This makes the design easier to extend — a future “VIPCustomer” could be added as one more subclass without touching any existing class.